

NVIDIA Omniverse 高階技術架構與實作全解析：從 OpenUSD 到生成式物理 AI 的數位雙生權威教材

現代工業數位化、機器人訓練以及複雜系統的數位雙生 (Digital Twin) 建構，正經歷一場底層架構的典範轉移。NVIDIA Omniverse 並非傳統意義上的「3D 建模軟體」(如 Autodesk Maya 或 Blender)，而是一個高度可擴展、基於 Universal Scene Description (OpenUSD) 框架构建的即時運算與協作平台¹。該平台透過整合高保真物理引擎、生成式人工智慧 (Generative AI) 以及分散式資料同步機制，為下一代物理 AI (Physical AI) 的開發提供了完整的基礎設施³。

本教材旨在為具備一定 3D 基礎的開發者與工程師，提供一份窮盡細節、兼具理論深度與實作邏輯的權威知識統整。內容將從 OpenUSD 的核心組合引擎出發，深入解析 3D 環境的生成與組裝標準、PhysX 5 的進階數學力學模型，並最終指導如何透過 Python 與 OmniGraph 建立受即時物聯網 (IoT) 數據驅動的數位雙生系統。

1. 核心概念：Omniverse 的底層架構與運作本質

要掌握 Omniverse，首要任務是解構其支撐多用戶、多應用程式無縫協作的三大技術支柱：OpenUSD 語言、Nucleus 伺服器架構，以及高度模組化的 Kit SDK。

1.1 OpenUSD 與 LIVERPS 組合引擎的深度解析

OpenUSD 是 Omniverse 的靈魂。它遠不僅是一種檔案儲存格式，而是一種先進的「場景描述語言」與高階的資料組合引擎¹。在傳統的 3D 工作流程中，資料的編輯通常是破壞性的 (Destructive)；然而，OpenUSD 允許不同部門的專家 (如幾何建模師、材質藝術家、燈光師與物理模擬工程師) 在各自獨立的圖層 (Layers) 中工作，並透過組合引擎將這些圖層非破壞性地疊加在一起，形成最終的場景圖 (Stage)²。

當多個資料來源對同一個幾何體 (Prim) 或屬性 (Property) 提供不同的「意見」(Opinions) 時，OpenUSD 依賴一套嚴謹的強度排序演算法來解決衝突。這套規則被稱為 **LIVERPS** 組合弧 (Composition Arcs)，其評估順序由最強至最弱排列，決定了最終呈現在場景中的資料狀態⁶。

組合弧 (Composition Arc)	代號	運作機制與架構意義
Local / Sublayers (本地與子圖層)	L	具備最高優先級的本地意見。子圖層採用深度優先 (Depth-first) 的順序進行解析，在圖層清單中排序較前

		的圖層，其資料強度將覆蓋排序較後的圖層，這是建構根圖層堆疊(Root Layer Stack)的核心機制 ⁶ 。
Inherits (繼承)	I	運作方式類似廣播機制。當多個目標 Prim 繼承自一個來源 Prim(通常帶有 Class 標述符)時，對來源的任何修改都會全域廣播並應用於所有繼承者。此機制強度高於參照，但可被本地圖層的意見所覆蓋 ⁶ 。
VariantSets (變體集)	V	允許在單一資產中封裝多種資料表示形式(如不同的幾何形狀或材質)，並可在執行階段動態切換。輕量級的變體切換僅改變屬性值，而重量級切換則會觸發場景結構的重新運算 ⁶ 。
rElocates (重定位)	E	OpenUSD 近期引入的組合弧，允許在不破壞原始參照結構與封裝性的前提下，動態更新與映射命名空間(Namespace)，有效解決複雜場景中的路徑衝突 ⁶ 。
References (參照)	R	用於將外部或內部的 Prim 階層嫁接至目前的場景中。參照過程中會自動進行路徑轉換(Path Translation)，使外部資產能完美融入新的命名空間，是實現資產模組化與大規模場景聚合的關鍵 ¹ 。
Payloads (負載)	P	在架構上與 References 完全相同，但額外提供了「延遲載入」(Deferred Loading)的能力。在處理如整座城市的巨型場景時，Payloads 允許

		系統在初始階段保持資產卸載狀態，藉此大幅節省記憶體並縮短載入時間 ⁶ 。
Specializes (特化)	S	強度最弱的組合弧。通常用於實現類似物件導向程式設計(OOP)中的預設值或備用值(Fallback)。任何由 Specializes 提供的屬性，皆可被其他任何組合弧輕易覆蓋 ⁶ 。

LIVERPS 演算法在解析場景時是遞迴(Recursive)執行的⁹。舉例而言，當系統解析一個 Payload 檔案時，會優先解析該檔案內部的 Local 意見，接著解析其內部的 Inherits 與 Variants，層層遞迴直到確定該資產的最終狀態，再將其整合至主場景中⁹。這種嚴密的遞迴與強度排序，確保了大規模協作時資料的絕對可預測性。

1.2 Nucleus Server 的協作與同步通訊架構

Omniverse Nucleus 是整個生態系的資料庫與協作引擎，負責管理 USD 資料的儲存與即時同步¹³。不同於傳統基於鎖定機制(File Locking)的網路硬碟，Nucleus 採用了先進的發布/訂閱(Publish/Subscribe)資料模型¹³。當客戶端應用程式(如 Blender 或 USD Composer)在本地端修改了場景結構，它會將差異資料(Deltas)發布至 Nucleus 伺服器，伺服器隨即將這些變更即時推播給所有已訂閱該場景的其他客戶端，實現極低延遲的即時協作(Live Sync)¹⁴。

Nucleus 企業版(Enterprise Nucleus Server)由多個高度容器化(Containerized)的微服務架構所組成：

- **Nucleus Core API Responder**：這是處理核心邏輯與暴露應用程式界面的主微服務，負責維護類似傳統檔案系統的樹狀資料目錄(以 / 分隔路徑)，但其底層實際上是關聯式與交易型資料庫的混合體¹⁶。
- **Large File Transfer (LFT) 服務**：專門為處理巨量二進位資料(如 8K 紋理貼圖、光達點雲資料)而設計的 HTTP 端點。在面臨高併發傳輸的企業環境中，LFT 服務可以水平擴展。架構指南建議每 10 個併發的上傳/下載需求，就應部署一個獨立的 LFT 容器實例¹⁷。
- **Discovery 與 Auth 服務**：Discovery 服務允許相鄰的微服務互相註冊與發現；Auth 服務則管理存取控制清單(ACLs)，並支援與外部身份提供者(如 Entra ID、Okta)整合進行 SAML 單一登入(SSO)¹³。

為了支撐龐大資料的瞬間吞吐量，特別是在超過 15 名使用者同時進行 Live Edit 的場景中(此時資料更新頻率呈現二次方級別增長)，底層硬體被強烈建議採用高速 NVMe 儲存設備，並確保極低的網路延遲¹⁸。

1.3 Kit SDK 的模組化擴充體系

所有 Omniverse 官方應用程式(包含 USD Composer、Isaac Sim、USD Explorer 等)本質上並非獨立開發的單體軟體,而是基於同一套 **Kit SDK** 載入不同擴充套件(Extensions)所拼裝出來的介面¹⁹。

Kit SDK 的底層依賴名為 Carbonite 的 C/C++ 框架,負責處理記憶體分配、多執行緒與作業系統抽象層²¹。在其之上,Kit 提供了強大的擴充套件系統,允許開發者以 Python 撰寫高階邏輯,並無縫呼叫底層編譯好的 C++ 函式庫²¹。每一個 Extension 都被嚴格封裝在獨立的資料夾中,其核心為一個名為 extension.toml 的設定檔²³。該檔案定義了套件的命名空間、相依性(Dependencies,例如依賴 "omni.kit.uiapp" 來渲染介面),以及 Python 模組的進入點²⁴。這種極致的模組化設計,使得開發者可以輕易攔截 Kit 的事件更新迴圈(Update Loop),將自定義的演算法或微服務注入到 Omniverse 生態系中¹⁹。

2. 3D 環境建模與生成式 AI workflow

在 Omniverse 的設計哲學中,針對基礎幾何體的創建採取了「連結(Connect)」大於「創造(Create)」的策略²⁵。平台本身不具備複雜的點/線/面網格編輯工具,而是依賴與外部生態系的深度整合,以及最新生成式 AI 的輔助。

2.1 Connectors 與雙向即時同步

傳統的數位內容創作(DCC)軟體,如 Blender、Autodesk Maya、3ds Max、Revit 與 Rhino,依然是幾何建模的主力²⁵。NVIDIA 開發了一系列的 Omniverse Connectors,這些外掛程式被安裝在各個 DCC 軟體內部,負責將軟體原生的資料結構(如 Maya 的 DAG 或 Blender 的 Mesh 數據)即時轉譯為 OpenUSD 格式²⁶。

透過 Nucleus 伺服器的橋接,這些 Connectors 建立了雙向即時同步(Live Sync)的通道。當建築師在 Rhino 中修改一面牆體的曲率,該變更會立刻被轉譯為 USD Deltas 傳送至 Nucleus,隨後 USD Composer 會在幾毫秒內更新其視窗中的幾何體²⁶。此工作流程打破了傳統「匯出再匯入」的孤島效應,確保了資料的唯一真值(Single Source of Truth)。

2.2 驅動物理 AI 的 Generative AI: Cosmos 與 Edify (2026 技術體系)

為了應對建構大規模數位雙生所需的海量資料,NVIDIA 正大力推動生成式 AI 進入 3D 場景。截至 2026 年,NVIDIA Cosmos 世界基礎模型(World Foundation Models, WFM)已成為合成資料生成的最新標準²⁸。Cosmos 平台具備深入理解物理定律的能力,其模型家族包含三大核心模組:

1. **Cosmos Predict 2.5 (未來狀態預測模型)**:這是一個基於流匹配(Flow-based)的先進模型,它將文字生世界(Text2World)、影像生世界(Image2World)與影片生世界(Video2World)統合成單一架構³⁰。結合 Cosmos Reason 1 視覺語言模型作為文字編碼器,Predict 2.5 能夠生成長達 30 秒、維持時空一致性與物理邏輯的高保真影片,極大程度地支援了機器人動作規劃與自駕車未來軌跡預測的訓練²⁹。
2. **Cosmos Transfer 2.5 (空間控制世界轉換模型)**:專為解決模擬到現實(Sim2Real)領域差距

而設計的 Multi-controlnet 模型²⁹。它能攝取來自 Isaac Sim 或 CARLA 等模擬器的結構化空間輸入(如深度圖、語意分割遮罩、邊緣特徵圖)，並套用 JSON 格式的 controlnet_specs 控制參數，將這些幾何骨架轉換為逼真且具備特定光照、天氣或背景條件的合成影片，且能保持多攝影機視角的一致性²⁹。

3. **Cosmos Reason 2 (具身智能推理模型)**: 專為物理 AI 打造的視覺語言推理模型，具備高達 256K 標記的長文本上下文理解能力³⁵。它利用思維鏈(Chain-of-Thought, CoT)技術分析物件互動與時空關係，甚至能執行精確的 2D 與 3D 空間座標定位及軌跡預測，讓 AI 代理能夠像人類一樣理解物理場景並評估動作的合理性³¹。

除此之外，**NVIDIA Edify** 架構負責直接生成靜態的 3D 資產與環境。透過整合如 Microsoft TRELIS 等技術的 3D 藍圖(Blueprints)，開發者僅需輸入文字提示(Prompt)，AI 便能在幾分鐘內生成拓撲完整、具備高解析度紋理的 3D 模型物件，或產生高達 16K 解析度的 360 度高動態範圍(HDRI)環境光照貼圖，徹底顛覆了傳統耗時的建模與打光流程³⁶。

2.3 MDL (Material Definition Language) 材質定義語言

要確保資產在不同渲染演算法(如即時光線追蹤 RTX Real-Time 與電影級路徑追蹤 Path Tracing)下呈現物理上絕對正確的視覺效果，Omniverse 採用了開源的 MDL 標準³⁸。MDL 將材質的光學行為與具體的渲染器實作分離。

在 USD Composer 中，開發者必須學習使用 **Material Graph**(節點式材質編輯器)來混階或自定義 MDL 雙向散射分佈函數(BSDFs)³⁸。針對高階應用，MDL 1.8 引入了如 anno::node_output_port_default 等標註(Annotations)，允許在節點圖中將複雜的單一結構體回傳值，智慧展開(Unrolling)為多個獨立的輸出端點，大幅簡化了材質節點的連接邏輯³⁹。更關鍵的是，開發者可利用 data_lookup 節點，讀取自訂的 USD Token 變數來即時驅動 MDL 的著色參數(Shading Parameters)，實現如「根據物體受損程度動態改變材質生鏽面積」的進階視覺反饋⁴⁰。

3. Environment 建立: 場景組裝與 SimReady 標準

環境的建立(Environment Creation)涵蓋了場景組裝(Scene Assembly)與世界建構(World Building)。這個階段主要在 **USD Composer** 中進行，是將零散的模型資產聚合成龐大且具備語意結構的虛擬世界的過程。

3.1 場景組裝與佈局工具

面對包含數以萬計物件的廠房或戶外環境，純手動拖拉模型是不切實際的。開發者需熟練運用幾項核心佈局擴充套件：

- **Array Tool 與 Distribute Tool**: 這些工具允許開發者設定特定的平移(Translate)與旋轉(Rotate)增量參數，將單一 USD 參照資產在 1D、2D 或 3D 空間中進行陣列複製與等距散佈，適用於組裝自動化產線或倉庫貨架⁴²。
- **Paint Tool**: 針對地形、植被等自然環境，開發者可以直接在拓撲網格上使用筆刷工具「畫」出資產。此工具底層利用射線投射(Raycasting)與幾何體實例化(Point Instancing)技術，能

以極低的記憶體開銷隨機生成大面積的森林或岩石群⁴⁴。

3.2 深入理解 SimReady (Simulation-Ready) 資產標準

在一般的 3D 渲染中，模型只需外觀精美即可；但在訓練感知 AI 與自動駕駛系統時，模型必須具備真實世界的物理屬性與中設資料。為此，NVIDIA 制訂了 **SimReady** 標準⁴。

要將建好的空殼模型轉化為合規的 SimReady 資產，必須滿足極其嚴格的技术規範：

1. 比例、對齊與拓樸：模型必須以「公尺」(Meters)為單位建立，保證真實世界比例。導出時的軸向必須設定為 **Z-up**，資產幾何中心須精確放置於原點 $(0, 0, 0)$ ，並且在正交前視圖中資產必須正確「朝前」⁴⁷。幾何體要求拓樸最佳化，避免浪費多邊形，且所有元素的命名 (Naming Convention) 必須清晰且具備唯一性，嚴禁出現如 Box01 或 pSphere27 此類無意義名稱⁴⁷。
2. **UV 佈局與材質**：UV 必須限定於 UV Channel 1 且絕對不可重疊 (Non-overlapping)。為了支撐近距離感測器模擬，UV 紋素密度 (Texel Density) 最低要求為每公尺 512 像素⁴⁷。材質方面，強制採用基於物理的 PBR Metal-Roughness 流程，紋理貼圖解析度上限通常為 4K 或 8K，且為了運算效能，單一物件強烈建議限制只綁定一個材質⁴⁷。
3. **語意標註 (Semantic Labeling)**：這是生成合成資料 (Synthetic Data Generation, SDG) 的命脈。模型必須透過 USDSemanticLabelsAPI 嵌入描述其屬性的元資料⁴⁸。為了避免命名歧義 (例如將同一種物體標註為 car 或 vehicle)，SimReady 規範採用開源的 **Wikidata Q-Codes** 作為統一的本體論 (Ontology)。每一個物件類型都對應一個全球唯一的字母數字 Q-Code，確保跨領域資料集的一致性⁴⁸。
4. **物理與感測器屬性**：模型必須掛載 USDPhysics Schema，設定精確的質量 (Mass)、重心、慣性張量，以及專門為物理碰撞優化的碰撞體 (Colliders，強烈建議避免單一複雜網格，改用多個 Convex Hulls 組成)⁴⁷。

3.3 Omni Asset Validator 與 DeepSearch

為確保資產嚴格遵循 SimReady 標準，Omniverse 提供了 omni.asset_validator.core 擴充套件⁵¹。開發者可透過 Python 腳本或 UI 介面載入 ValidationEngine，該引擎內含多種驗證規則：

- OmniDefaultPrimChecker 確保資產具備單一旦啟用的根節點⁵²。
- StageMetadataChecker 驗證 upAxis 和 metersPerUnit 設定是否正確⁵²。
- 針對特定領域的 VehicleCapabilityChecker 會檢查車輛資產是否對齊軸線、輪胎是否接觸地面，以及所有網格是否皆已正確分類為車體零件⁵³。若發現違規，引擎會拋出錯誤，並允許開發者透過 IssueFixer 模組套用自動修正建議⁵²。

在資產庫檢索方面，**DeepSearch** 技術摒棄了傳統的檔名搜尋⁵⁴。當資產匯入 Nucleus 時，渲染微服務會在背景從 8 個不同視角對 USD 資產進行拍攝，接著透過 Embedding 服務提取高維度的 NVCLIP 向量並儲存於 OpenSearch 資料庫中⁵⁴。這使得開發者能直接輸入自然語言提示 (如 "Red rusty car")，AI 即可透過語意與視覺特徵的比對，瞬間在龐大圖庫中精準找到目標物件⁵⁴。

4. Physics 模擬: PhysX 5 的動態數學模型與力學

在 Omniverse 建立的數位雙生中，物理不再依賴預先烘焙 (Baked) 的動畫關鍵格，而是由 **PhysX 5** 核心引擎執行的即時微分方程求解⁵⁵。

4.1 剛體動力學 (Rigid Body Dynamics) 與碰撞最佳化

剛體物理用於處理形狀不可改變的物件 (如金屬連桿、木箱)。要實現穩定且高效的剛體模擬，開發者必須學會調校物理求解器 (Solver) 設定⁵⁷。

在複雜場景中，可以透過 `PhysxSchema.PhyxSceneAPI` 限制全局的迭代次數以換取效能。求解器演算法的選擇至關重要：雖然投影高斯-賽德爾 (Projected Gauss-Seidel, PGS) 演算法在單次迭代中速度較快，但強烈建議採用 時間高斯-賽德爾 (**Temporal Gauss-Seidel, TGS**) 求解器，因為 TGS 在較少的迭代次數下能提供更卓越的收斂性與物理穩定性⁵⁷。

碰撞調校 (**Collision Tuning**) 是新手與專家的分水嶺。直接使用高面數的視覺網格 (Render Mesh) 作為碰撞體會導致模擬極度緩慢甚至穿模。專家級的工作流會採用凸分解 (Convex Decomposition) 技術，將複雜的凹多邊形拆解為一組近似的凸包 (Convex Hulls) 集合；或者利用最新支援的符號距離場 (Signed Distance Field, SDF) 碰撞器，免分解直接模擬齒輪等複雜凹面物件的精準咬合⁵⁵。

4.2 軟體動力學 (Soft Body Dynamics) 與 FEM 數學解析

針對肉體、橡膠、布料等受力後會產生形變的物體，PhysX 5 引入了強大的有限元素法 (**Finite Element Method, FEM**) 進行求解⁵⁹。

FEM 模型揚棄了表面多邊形，而是將物體內部離散化為立體的四面體網格 (Tetrahedral Meshes)⁵⁹。在架構上，系統維護兩套網格：一套是低解析度的「模擬網格」(Simulation Mesh) 用於快速計算力學變形，另一套則是緊密貼合視覺網格的高解析度「碰撞網格」(Collision Mesh) 以確保碰撞反應的精準度⁵⁹。

其底層採用「固定共旋線性彈性模型」(Fixed Corotated Linear Elasticity Model)，開發者主要透過 `PxFEMMaterial` 介面調校兩個關鍵物理參數⁵⁹：

1. 楊氏模數 (**Young's Modulus, E**)：定義物體的整體剛度與抗形變能力。數值越大，物體越堅硬⁵⁹。
2. 柏松比 (**Poisson's Ratio, ν**)：定義物體的體積守恒性 (不可壓縮性)，其有效範圍嚴格限制在 $[0, 0.5]$ 之間⁵⁹。當數值越接近 0.5 時，物體越具備不可壓縮的特性 (例如水球)⁵⁹。

PhysX 求解器會將這兩個工程參數轉換為拉梅參數 (Lamé Parameters) 來計算能量密度函數⁵⁹：

- 第一拉梅參數 μ 負責抵抗拉伸與剪切應力，與「盡可能保持剛性」(ARAP) 的能量項相關：

$$\mu = \frac{E}{2(1+\nu)}$$

- 第二拉梅參數 λ 控制體積守恒的能量項： $\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}$

由於高楊氏模數的極硬軟體會導致求解器極難收斂，開發者除了降低時間步長與增加迭代次數外，還必須利用 `setDamping()` 函數設定阻尼，抑制高頻震盪以避免模擬在視覺上發生爆炸 (Numerical Explosion) ⁵⁹。

4.3 破壞模擬 (Blast) 與應力損傷模型

對於物體的碎裂與爆炸，omni.blast 擴充套件整合了 NVIDIA Blast SDK ⁶³。與過去依賴腳本觸發預設動畫不同，最新的 Blast 系統導入了真實的應力損傷系統 (Stress Damage System) ⁶⁵。

Blast 在內部將預先切割好的碎片建立一個無形的「支撐圖」(Support Graph)，節點之間透過數學鍵 (Bonds) 相連 ⁶⁴。在模擬過程中，物理引擎會計算動態碰撞所產生的脈衝力，並透過圖結構傳遞應力。當局部應力超過該鍵結設定的結構強度極限時，鍵結斷裂，碎片成為獨立的剛體剝離主體 ⁶⁵。多餘的動能會自動轉化為碎片的初始速度，形成極其真實的爆破飛散軌跡。開發者可藉由微調 `bondIterationsPerFrame` 參數，在裂痕運算的精細度與 CPU 效能之間取得平衡 ⁶⁵。

4.4 機器人與車輛動力學 (Robotics & Vehicle Dynamics)

在 Isaac Sim 的機器人模擬中，傳統的極大座標關節 (Maximal Coordinate Joints) 容易在串聯複雜的機械手臂時產生關節漂移。因此必須採用縮減座標關節 (Reduced Coordinate Articulations)，它能在線性時間內保證關節無誤差 (Joint-error-free)，是訓練強化學習 (RL) 模型的必備設定 ⁵⁵。針對容易產生數值不穩定的閉環運動鏈 (Closed Loops) 或高阻力抓取動作，專家技巧是為驅動器 (Drives) 或模擬關節 (Mimic Joints) 添加微小的柔性 (Compliance) 與阻尼比，透過允許極小幅度的物理讓步，化解求解器中的硬約束衝突 ⁶⁷。

針對車輛模擬，omni.physx.vehicle 提供了一套專用模組，能即時運算輪胎摩擦力、離合器傳動，以及由「懸吊最大行程」和「彈簧強度」所驅動的動態懸吊系統。開發者甚至能啟用阿克曼轉向幾何 (Ackermann Steering Correction)，確保車輛在轉彎時內側輪的轉向角度大於外側輪，避免車胎產生不真實的橫向滑動 ⁶⁸。

5. 邏輯自動化與數位雙生整合實戰 (Action Plan)

建構出擁有物理法則的 3D 環境後，下一步是賦予其「生命」，也就是讓場景具備互動邏輯，甚至連接真實世界的感測器資料。

5.1 視覺化腳本：深入 OmniGraph 與 Action Graphs

Omniverse 的邏輯控制核心是 OmniGraph，它是一種視覺化節點程式系統 (類似 Unreal 的 Blueprints) ⁷⁰。在各種圖形中，Action Graphs (動作圖) 專門用於處理事件驅動 (Event-driven) 的互動行為 ⁷¹。

理解 Action Graphs 的關鍵在於掌握「執行流」(Execution Flow)。每一個動作圖都必須以「事件節點」(Event Source Nodes)作為起點，例如 On Playback Tick(在模擬播放期間每影格觸發)或 On Keyboard Input⁷¹。這些事件節點會發出執行脈衝(Execution Token)，沿著節點連線向下游傳遞。

為達成複雜的邏輯判斷，開發者需精通 流程控制節點 (Flow Control Nodes)⁷¹：

- **Branch (分支)**：相當於程式中的 if-else。讀取一個布林值條件，若為真則驅動一條執行路徑，若為假則驅動另一條⁷¹。
- **Flip Flop (觸發器)**：每次接收到執行脈衝時，會交替觸發 A 輸出與 B 輸出。非常適合用於按鈕開關、燈光切換等單一事件切換兩種狀態的場景⁷³。
- **Sequence (序列) 與 Delay (延遲)**：Sequence 能將單一輸入的執行脈衝依序拆分為多個後續執行動作；Delay 則能暫停執行流指定的 Tick 數或時間，用於製作排程動畫或等待感測器反饋⁷¹。

進階開發者可透過定義 JSON 格式的 .ogn 檔案，結合 C++ 或 Python API(如 omni.graph.action.IActionGraph)，編寫完全客製化的運算節點⁷¹。

5.2 Python Scripting 與 Extension 開發

雖然節點介面直觀，但 Omniverse 真正的自動化力量在於 Python 腳本(Omni.kit API)⁷⁷。要讓工作流自動化，開發者應學習撰寫獨立的 Extension¹⁹。

一個標準 Python Extension 的結構始於根目錄下的 config/extension.toml²³。開發者必須在該 TOML 檔中宣告模組名稱 [[python.module]] 並載入必要的 UI 依賴(如 "omni.kit.uiapp" = {})²⁴。在原始碼資料夾中，__init__.py 負責定義套件的生命週期，透過實作 on_startup() 與 on_shutdown() 方法，在使用者啟用套件時自動生成 omni.ui 使用者介面，或動態讀取、修改 USD Stage 內的屬性節點⁷⁸。

5.3 數位雙生實戰：導入真實世界 IoT 數據驅動模型

將真實世界的感測器(如自動無人搬運車的貨叉高度、馬達轉速)對接到 Omniverse 中，是實踐工業數位雙生的最終里程碑⁸⁰。

目前的標準作法是開發一個無頭(Headless)的 Connector Extension，透過 MQTT 協定 建立發布/訂閱連線⁸¹。在 extension.toml 中宣告 paho-mqtt 依賴後，Python 腳本會建立 MQTT Client，並訂閱特定主題(如 synctwin/forklift/fork_level)⁸¹。

關鍵技術瓶頸與解法 (Thread-Safety)：當 MQTT 接收到非同步的網路封包時，回呼函式(Callback)通常執行於 Python 的背景執行緒中。然而，OpenUSD 的核心架構在寫入操作上並非執行緒安全(**Not Thread-Safe**)⁸³。若直接在網路執行緒中呼叫 Usd.Attribute.Set() 嘗試移動 3D 模型，將會引發 Python 的全域直譯器鎖(GIL)衝突與 USD 資料庫毀損⁸³。

正確的專家級實作方式是：在 MQTT 的 on_message 執行緒中，僅將接收到的數值快取至記憶體

中的資料模型 (Data Model)⁸¹。隨後, Extension 必須向 Omniverse 註冊並訂閱主執行緒的更新事件串流 (**update_event_stream**)。當主程式進入每個影格的 Tick 安全更新週期時, 主執行緒內的 Callback 才會去讀取該快取數值, 並安全地寫入 USD 屬性中, 從而物理正確且平滑地驅動虛擬堆高機的貨叉升降⁸¹。對於包含數以萬計感測器的極大規模工廠數位雙生, 更進階的架構會繞過傳統 USD, 直接呼叫 **USDRT / Fabric API**。Fabric 提供了一個向量化的高速運行時快取, 允許直接對 GPU/CPU 資料結構進行高效修改, 確保畫面更新率不受海量 IoT 寫入事件影響⁸³。

引用的著作

1. References — Learn OpenUSD, 檢索日期: 2月 21, 2026, <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/what-are-references.html>
2. Composition Basics — Learn OpenUSD, 檢索日期: 2月 21, 2026, <https://docs.nvidia.com/learn-openusd/latest/composition-basics/index.html>
3. Develop Physical AI Applications | NVIDIA Omniverse, 檢索日期: 2月 21, 2026, <https://www.nvidia.com/en-us/omniverse/>
4. What Is SimReady? | NVIDIA Glossary, 檢索日期: 2月 21, 2026, <https://www.nvidia.com/en-us/glossary/simready/>
5. USD Quickstart Guide - OpenUSD, 檢索日期: 2月 21, 2026, https://openusd.org/files/USD_Quickstart_Guide.pdf
6. LIVERPS Strength Ordering — Learn OpenUSD, 檢索日期: 2月 21, 2026, <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/index.html>
7. What Is LIVERPS? — Learn OpenUSD, 檢索日期: 2月 21, 2026, <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>
8. OpenUSD Variants and Strength Ordering (LIVERPS) | Learn With Me - YouTube, 檢索日期: 2月 21, 2026, <https://www.youtube.com/watch?v=USOtmAaRjUM>
9. Composition Arcs and Strength Ordering — Learn OpenUSD, 檢索日期: 2月 21, 2026, <https://docs.nvidia.com/learn-openusd/latest/composition-basics/strength-ordering.html>
10. References and Payloads — Learn OpenUSD, 檢索日期: 2月 21, 2026, <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/index.html>
11. Composition Strength Ordering (LIVERPS) - Usd Survival Guide, 檢索日期: 2月 21, 2026, <https://lucascheller.github.io/VFX-UsdSurvivalGuide/pages/core/composition/liverps.html>
12. USD Frequently Asked Questions — Universal Scene Description 26.03 documentation, 檢索日期: 2月 21, 2026, <https://openusd.org/dev/usdfaq.html>
13. Nucleus Overview - NVIDIA Omniverse, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/nucleus/latest/index.html>
14. Features and Benefits — Omniverse Nucleus, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/nucleus/latest/features.html>

15. How can I connect multiple computers working on different programs through omniverse, 檢索日期: 2月 21, 2026, <https://forums.developer.nvidia.com/t/how-can-i-connect-multiple-computers-working-on-different-programs-through-omniverse/193444>
16. Architecture — Omniverse Nucleus, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/nucleus/latest/architecture.html>
17. Enterprise FAQ - Base Nucleus Stack - NVIDIA Omniverse, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/nucleus/latest/enterprise/enterprise-faq.html>
18. Hardware Sizing Guide — Omniverse Nucleus, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/nucleus/latest/sizing-guide.html>
19. Kit Architecture — Omniverse Developer Guide, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/dev-guide/latest/kit-architecture.html>
20. The Omniverse Kit SDK — kit-app-template, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/kit/docs/kit-app-template/latest/docs/kit_sdk_overview.html
21. Omniverse Physics Developer Guide, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/index.html
22. Overview — Omniverse Kit, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/kit/docs/kit-manual/latest/guide/kit_overview.html
23. Extensions in-depth — Omniverse Kit, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/kit/docs/kit-manual/latest/guide/extensions_advanced.html
24. Configure TOML Files — Omniverse Developer Guide, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/dev-guide/latest/dev_guide/develop/develop_configure_toml_files.html
25. Third Party Connectors — Omniverse Connect, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/connect/latest/3rd-party-connectors.html>
26. Live Sync — Omniverse Connect, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/connect/latest/rhino/manual/live.html>
27. Live Workflow — Omniverse Connect, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/connect/latest/sketchup/manual/live.html>
28. Physical AI with World Foundation Models | NVIDIA Cosmos, 檢索日期: 2月 21, 2026, <https://www.nvidia.com/en-us/ai/cosmos/>
29. Cosmos Predict 2.5 & Transfer 2.5: Evolving the World Foundation Models for Physical AI, 檢索日期: 2月 21, 2026, <https://huggingface.co/blog/nvidia/cosmos-predict-and-transfer2-5>
30. Cosmos-Predict2.5, the latest version of the Cosmos World Foundation Models (WFMs) family, specialized for simulating and predicting the future state of the world in the form of video. - GitHub, 檢索日期: 2月 21, 2026, <https://github.com/nvidia-cosmos/cosmos-predict2.5>
31. Scale Synthetic Data and Physical AI Reasoning with NVIDIA Cosmos World Foundation Models, 檢索日期: 2月 21, 2026, <https://developer.nvidia.com/blog/scale-synthetic-data-and-physical-ai-reasoning-with-nvidia-cosmos-world-foundation-models/>

32. GitHub - nvidia-cosmos/cosmos-transfer2.5, 檢索日期: 2月 21, 2026,
<https://github.com/nvidia-cosmos/cosmos-transfer2.5>
33. Introduction — Cosmos, 檢索日期: 2月 21, 2026,
<https://docs.nvidia.com/cosmos/latest/introduction.html>
34. How to Scale Data Generation for Physical AI with the NVIDIA Cosmos Cookbook, 檢索日期: 2月 21, 2026,
<https://developer.nvidia.com/blog/how-to-scale-data-generation-for-physical-ai-with-the-nvidia-cosmos-cookbook/>
35. NVIDIA Cosmos Announcements at CES 2026, 檢索日期: 2月 21, 2026,
<https://forums.developer.nvidia.com/t/nvidia-cosmos-announcements-at-ces-2026/356629>
36. NVIDIA Researchers Harness Real-Time Gen AI to Build Immersive Desert World, 檢索日期: 2月 21, 2026,
<https://blogs.nvidia.com/blog/real-time-3d-generative-ai-research-siggraph-2024/>
37. 3D Object Generation Blueprint by NVIDIA, 檢索日期: 2月 21, 2026,
<https://build.nvidia.com/nvidia/object-generation-3d>
38. Material Graph — Omniverse Extensions, 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/extensions/latest/ext_material/ext_material-graph.html
39. MDL Language — Omniverse Materials and Rendering, 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/materials-and-rendering/latest/mdl_language.html
40. Omniverse Materials Workflows, 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/materials-and-rendering/latest/materials_workflows.html
41. Using custom .mdl files on Material Graph Editor - NVIDIA Developer Forums, 檢索日期: 2月 21, 2026,
<https://forums.developer.nvidia.com/t/using-custom-mdl-files-on-material-graph-editor/305564>
42. Array Tool — Omniverse Extensions, 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/extensions/latest/ext_array-tool.html
43. Distribute Tool — Omniverse Extensions, 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/extensions/latest/ext_distribute-tool.html
44. Design — Omniverse Extensions, 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/extensions/latest/ext_design.html
45. Extensions Overview - NVIDIA Omniverse, 檢索日期: 2月 21, 2026,
<https://docs.omniverse.nvidia.com/extensions/latest/index.html>
46. The SimReady Specification - NVIDIA Omniverse, 檢索日期: 2月 21, 2026,
<https://docs.omniverse.nvidia.com/simready/latest/overview/simready-spec.html>
47. SimReady 3D Art Asset Creation - NVIDIA Omniverse, 檢索日期: 2月 21, 2026,
<https://docs.omniverse.nvidia.com/simready/latest/simready-asset-creation.html>
48. SimReady Semantic Labelling - NVIDIA Omniverse, 檢索日期: 2月 21, 2026,
<https://docs.omniverse.nvidia.com/simready/latest/sim-needs/semantic-labeling.html>

49. Semantic Labels — Requirement Documentation - NVIDIA Omniverse, 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/kit/docs/asset-requirements/1.7.1/capabilities/semantic_labels/capability-semantic_labels.html
50. New Cloud Applications, SimReady Assets, and Tools for NVIDIA Omniverse Developers Announced at GTC, 檢索日期: 2月 21, 2026,
<https://developer.nvidia.com/blog/new-cloud-applications-simready-assets-and-tools-for-omniverse-developers-announced-at-gtc/>
51. Omni Asset Validator (Tutorial), 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/kit/docs/asset-validator/0.25.3/source/extensions/omni.asset_validator.core/docs/tutorial/index.html
52. Omni Asset Validator (Tutorial), 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/kit/docs/asset-validator/latest/source/extensions/omni.asset_validator.core/docs/tutorial/index.html
53. » Omni Asset Validator (SimReady) » Rules - NVIDIA Omniverse, 檢索日期: 2月 21, 2026,
<https://docs.omniverse.nvidia.com/kit/docs/asset-validator/0.20.0/source/src/docs/simready/rules.html>
54. Architecture — Omniverse Services, 檢索日期: 2月 21, 2026,
<https://docs.omniverse.nvidia.com/services/latest/services/usd-search/architecture.html>
55. PhysX SDK - Latest Features & Libraries | NVIDIA Developer, 檢索日期: 2月 21, 2026, <https://developer.nvidia.com/physx-sdk>
56. Announcing NVIDIA PhysX SDK 5.0 | NVIDIA Technical Blog, 檢索日期: 2月 21, 2026,
<https://developer.nvidia.com/blog/announcing-nvidia-physx-sdk-5-0/>
57. Simulation Performance Guide — Omni Physics - NVIDIA Omniverse, 檢索日期: 2月 21, 2026,
https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/guides/physics-performance.html
58. Rigid Body Collision — physx 5.1.0 documentation, 檢索日期: 2月 21, 2026,
<https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/docs/RigidBodyCollision.html>
59. Soft Bodies — physx 5.4.1 documentation, 檢索日期: 2月 21, 2026,
<https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/SoftBodies.html>
60. PxFEMMaterial — physx 5.3.1 documentation - GitHub Pages, 檢索日期: 2月 21, 2026,
https://nvidia-omniverse.github.io/PhysX/physx/5.3.1/_api_build/class_px_f_e_m_material.html
61. PxFEMSoftBodyMaterial — physx 5.3.0 documentation - GitHub Pages, 檢索日期: 2月 21, 2026,
https://nvidia-omniverse.github.io/PhysX/physx/5.3.0/_api_build/class_px_f_e_m_soft_body_material.html
62. Deformable physics object - Poisson's ratio, Young's modulus - Isaac Sim, 檢索日期: 2月 21, 2026,
<https://forums.developer.nvidia.com/t/deformable-physics-object-poissons-ratio-youngs-modulus/308745>

63. Blast Destruction — omni.blast 0.11.15 documentation - NVIDIA Omniverse, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/kit/docs/omni.blast>
64. blast-sdk 5.0.6 documentation - GitHub Pages, 檢索日期: 2月 21, 2026, <https://nvidia-omniverse.github.io/PhysX/blast/index.html>
65. NVIDIA(R) Blast(R) SDK 1.1 API Reference: Stress Solver (NvBlastExtStress), 檢索日期: 2月 21, 2026, https://docs.nvidia.com/gameworks/content/gameworkslibrary/blast/1.1/api_docs/files/pageextstress.html
66. Set damage amount for slow moving rigid body - NVIDIA Developer Forums, 檢索日期: 2月 21, 2026, <https://forums.developer.nvidia.com/t/set-damage-amount-for-slow-moving-rigid-body/216060>
67. Articulation and Robot Simulation Stability Guide — Omni Physics - NVIDIA Omniverse, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/guides/articulation_stability_guide.html
68. Vehicle Dynamics Visual Authoring — Omni Physics, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/vehicles/vehicles.html
69. Untangling the Nvidia Omniverse PhysX Vehicle SDK workflow | by SpicyTech - Medium, 檢索日期: 2月 21, 2026, <https://jmake.medium.com/untangling-the-nvidia-omniverse-physx-vehicle-sdk-workflow-643e07b3757b>
70. OmniGraph — Omniverse Extensions, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/extensions/latest/ext_omnigraph.html
71. Action Graph — kit-omnigraph - NVIDIA Omniverse, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/kit/docs/omni.graph.docs/latest/concepts/ActionGraph.html>
72. Introduction to OmniGraph — Omniverse Extensions, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/extensions/latest/ext_omnigraph/tutorials/gentle_intro.html
73. Using Flow Control Nodes — Omniverse Extensions, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/extensions/latest/ext_omnigraph/tutorials/using_flow_control_nodes.html
74. Action Graph Code Samples - Python — kit-omnigraph - NVIDIA Omniverse, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/kit/docs/omni.graph.docs/latest/dev/ogn/action_code_samples_python.html
75. OmniGraph Nodes - NVIDIA Omniverse, 檢索日期: 2月 21, 2026, <https://docs.omniverse.nvidia.com/kit/docs/omni.graph.docs/latest/dev/WritingNodes.html>
76. OGN User Guide — kit-omnigraph - NVIDIA Omniverse, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/kit/docs/omni.graph.docs/latest/dev/ogn/ogn_user_guide.html
77. Python Scripting Component - Getting Started — Omniverse Extensions, 檢索日期: 2月 21, 2026, https://docs.omniverse.nvidia.com/extensions/latest/ext_omnigraph/tutorials/python_scripting_component_getting_started.html

期:2月 21, 2026,

https://docs.omniverse.nvidia.com/extensions/latest/ext_python-scripting-component/getting_started.html

78. Directory Structure — kit-omnigraph - NVIDIA Omniverse, 檢索日期:2月 21, 2026

<https://docs.omniverse.nvidia.com/kit/docs/omni.graph.docs/latest/dev/DirectoryStructure.html>

79. Navigating Extensions from Other Developers — Omniverse Workflows, 檢索日期:2月 21, 2026,

https://docs.omniverse.nvidia.com/workflows/latest/extensions/julia_modeler.html

80. Welcome to Assembling Digital Twins With Omniverse and OpenUSD, 檢索日期:2月 21, 2026, <https://docs.omniverse.nvidia.com/digital-twins/latest/index.html>

81. How to control a digital twin asset with MQTT in NVIDIA Omniverse | by Michael T. Wagner, 檢索日期:2月 21, 2026,

<https://mtw75.medium.com/how-to-control-a-digital-twin-asset-with-mqtt-in-nvidia-omniverse-92382e92e4dc>

82. README.md - NVIDIA-Omniverse/iot-samples - GitHub, 檢索日期:2月 21, 2026,

<https://github.com/NVIDIA-Omniverse/iot-samples/blob/main/README.md>

83. Use multiple threads in Omniverse - General Discussion - NVIDIA Developer Forums, 檢索日期:2月 21, 2026,

<https://forums.developer.nvidia.com/t/use-multiple-threads-in-omniverse/249989>

84. Multi-threading in Omniverse extensions - General Discussion - NVIDIA Developer Forums, 檢索日期:2月 21, 2026,

<https://forums.developer.nvidia.com/t/multi-threading-in-omniverse-extensions/236553>

85. Create an Attribute — Omniverse Developer Guide, 檢索日期:2月 21, 2026,

https://docs.omniverse.nvidia.com/dev-guide/latest/programmer_ref/usd/properties/create-attribute.html

86. USD, Fabric, and USDRT - NVIDIA Omniverse, 檢索日期:2月 21, 2026,

https://docs.omniverse.nvidia.com/kit/docs/usdrt/latest/docs/usd_fabric_usdrt.html